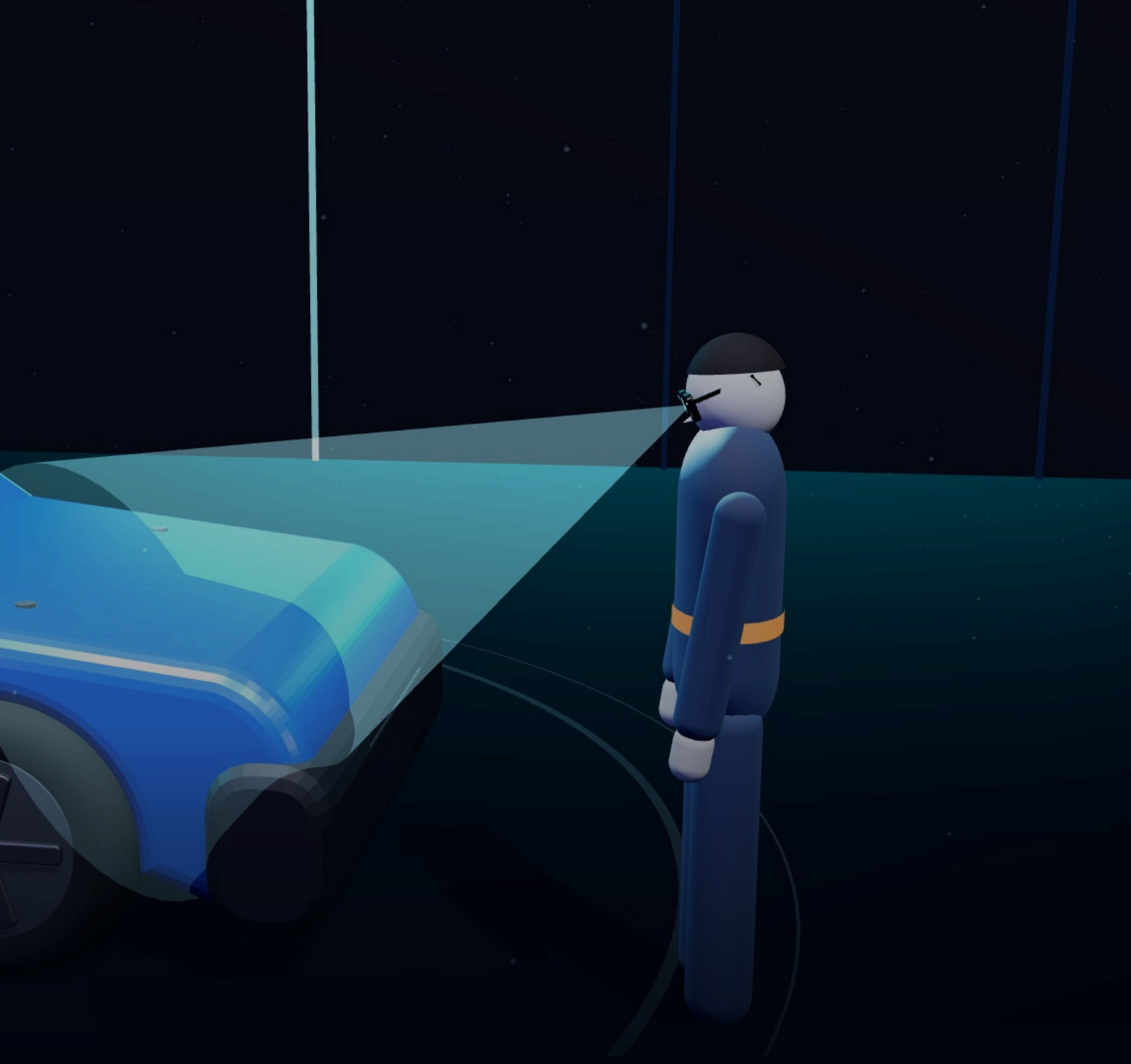


USTAD

The master technician in your glasses

Project guide: how every feature works, how to install and use the app, and how to run the demo

Meta Wearables Studio UAE · Android app on the Meta Wearables Device Access Toolkit 1.0.1
App version **2.7** · Repository **github.com/Shaaha7/meta-1** · Demo film **docs/demo/ustad-demo.mp4**

Contents

1. What Ustad is, in one page
2. How it works: from the glasses to the answer
3. Every feature, how it works and how to use it
4. Install the app
5. Using the app: with no glasses, with Mock Device Kit, with real glasses
6. The demo film: scene by scene, how to play and remake it
7. Live demo script for judges (3 minutes)
8. Troubleshooting
9. Technology and project layout

1 What Ustad is

Ustad (“master” in Arabic, Urdu and Hindi) turns Meta Ray-Ban / Oakley Meta glasses into a senior technician who stands next to you. You look at a machine and ask. Ustad sees the problem, explains why it happens, shows the hidden failing part in a 3D exploded view and talks you through the fix, hands-free.

🔍 Inspect

Look at a car, an AC or any machine. Ustad finds the defect, ranks the likely causes and shows the failing part inside the machine in 3D.

🎓 Teach

An expert wears the glasses and does the job once. Ustad turns the recording into a lesson (steps, why, safety, checkpoints, quiz) and the expert approves it.

👩‍🔧 Learn

A learner is coached step by step in their own language. Ustad checks their work by photo and records the result in a Skill Passport.

Why it can be trusted

- **Privacy first:** faces and number plates are blurred on the phone before anything is sent. If the blur fails, nothing is sent.
- **Not just an AI guess:** for cars and ACs, Ustad's own knowledge-base brain ranks the causes with Bayesian probability and cites sources. The AI only reports what it can see.
- **Proof:** each cause comes with a 10-second test, and the result is checked from a photo.
- **Safety Stop:** fixed rules check every AI repair step. Dangerous work (high-voltage EV, refrigerant, gas, airbags, mains panels) stops with “call a certified professional”.
- **Experts stay in charge:** lessons are published only after an expert approves them.

2 How it works

One inspection, from the moment the technician speaks to the moment the part is shown:

1 · Ask — "Mentor, inspect" (your wake word) or the capture button on the glasses

→

2 · Capture — The glasses camera takes a photo through the Device Access Toolkit

→

3 · Privacy — ML Kit finds faces and plates on the phone and pixelates them

→

4 · See — Gemini (or Llama offline) reports the machine and the symptoms it sees

→

5 · Reason — The brain ranks causes from the knowledge graph and picks the next question

→

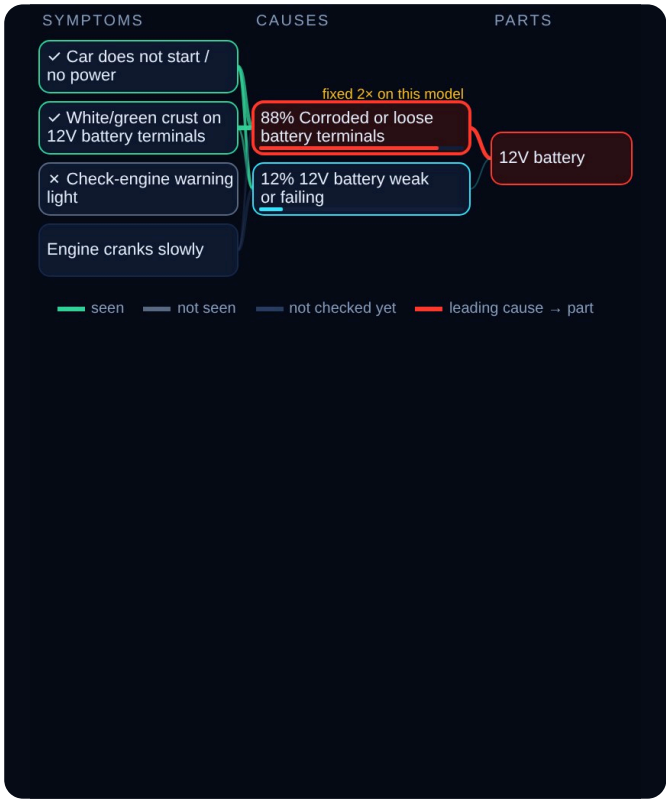
6 · Show — 3D exploded view, voice answer, steps on the lens or phone

The machine router

After the AI names the machine, Ustad routes it: **car** and **split AC** go to their knowledge bases and 3D models, a **lithium battery pack** goes to the battery module, and **anything else** gets an AI diagnosis with an AI-drawn 3D layout. Matching uses whole words, so "tractor" is never mistaken for "AC".

The brain (knowledge-base diagnosis)

- **Knowledge graph:** symptom → failure mode → internal part. Each link stores how likely the symptom is under that failure, plus a quick test, fix steps and sources (`assets/ustad/kb/car.json`, `split_ac.json`).
- **Ranking:** P(failure | evidence) ∝ prior × Π P(symptom | failure).
- **Next best question:** the unanswered symptom with the highest information gain ("Does it crank slowly?").
- **Check in this order:** checks sorted by likelihood per minute of testing, so the quick likely causes come first.
- **Explains itself:** how strongly each piece of evidence supports or argues against the top cause.
- **Learns:** each confirmed fix updates the priors (the knowledge base counts as 20 cases; a fix on the same make and model counts 4×). Learning can be exported and imported between phones.



The reasoning graph on the results screen: evidence → causes, line thickness = strength.

3 Every feature

Each card: what it does, how it works inside, and how to use it (**Use:**). “Hub” means the in-app USTAD screen with tabs Inspect · Learn · Team · Settings.

🔍 Inspect + X-ray reveal

A photo of a machine becomes a diagnosis: what the machine is, what is wrong, the most likely causes with percentages, and the hidden failing part in a 3D exploded view.

Use: Hub → Inspect → Take a photo / From gallery. With glasses: say “*word*, inspect” or press capture.

🧊 3D exploded view

three.js models of a car (16 parts: engine, 12V battery, radiator, brakes, EV battery…), a split AC and a battery pack. The failing part pulses red, everything else turns ghost-like. Drag to rotate, pinch to zoom, tap Explode / Assemble.

Use: opens automatically in the result (“Inside view”).

🧠 Brain & reasoning graph

Cars and ACs are ranked by Ustad's own Bayesian model, not the AI. The result shows the top causes, the next question to answer, the check order and a graph of how it decided.

Use: results screen → More details → “How Ustad decided”. Answer the questions by tapping or saying yes / no / not sure.

🧪 Prove it

Each cause comes with a 10-second test (“wiggle each terminal clamp”). Take a photo of the result; the AI judges only that photo: confirmed (the brain learns), not confirmed (next cause) or unclear (what to show instead).

Use: results → Prove it → 📸, or say “*word*, check”.

🎧 Listen

Records 6 seconds of the running machine. The phone measures loudness, pitch and spectrum; the AI marks sound symptoms (rattle, hiss, squeal, slow crank, rough idle) as evidence for the brain.

Use: results → More details → Listen to the machine.

🛡 Safety Stop

Fixed rules (not AI) check repair steps and lesson drafts. If step 1 doesn't make the machine safe, an isolation step is added. High voltage, refrigerant, gas, airbag and mains work stops the coaching; engines, fuel, capacitors, jacks and hot parts get warnings.

Use: automatic; shown as a yellow SAFETY STOP.

🔧 Repair coach

Speaks the fix one step at a time in English, Hindi or Arabic (steps translated by the AI). Move with voice: next, back, repeat, why.

Use: results → Start repair coach.

🔋 Battery module

Lithium packs (e-bike, scooter, power bank, tools). Ustad reads the label and looks for swelling, leaks, burns and corrosion; you add what you measured. A rule model gives OK / WATCH / REPLACE / ISOLATE NOW, remaining cycles (NASA Ames ageing data: 70% capacity = end of life) and the failing layer in a 3D cell.

Use: Hub → Inspect → Battery check, or inspect a photo of a pack.

🎓 Teach (lesson builder)

The expert's video is cut into about 10 face-blurred key frames, and their voice is kept. The AI drafts a lesson: steps, *why* each matters, safety, photo checkpoints, a quiz and the 3D part for each step, with the expert's own words per step. The expert edits and approves it.

Use: Hub → Learn → Record an expert / Pick a video → Build lesson → Approve & publish. With glasses: "*word*, start lesson" … "*word*, stop lesson".

👩‍🔧 Learn (guided lesson)

The learner hears each step, sees the part in 3D, sends a photo at checkpoints for Ustad to check, then takes the quiz. Two built-in car lessons (12V battery terminals, tire check) work with **no AI at all**, so they are a safe demo path.

Use: Hub → Learn → pick a lesson → Start.

🏅 Skill Passport

Score = 60% quiz + 40% checkpoints passed first time. A score of 80+ earns a skill badge.

Use: Hub → Learn → Skill Passport.

📊 Team dashboard

Technicians, badges, who needs help, the most common faults and recent jobs. Records move between phones as a file; the dashboard is shared as one HTML page anyone can open.

Use: Hub → Team → Open dashboard / Share.

📞 Live expert

Sends a job summary (machine, cause, what was tried) to a senior on WhatsApp, then you start a glasses video call: "Hey Meta, video call … on WhatsApp".

Use: Hub → Team → Your expert (set name and number), then "*word*, call expert" or the button in the result.

🕶 Ray-Ban Display

On display glasses, the lens shows the cause or the current step, the exploded view with the part in red and one line of instruction (mwdat-display). Glasses without a display simply decline.

Use: automatic when the glasses session starts; the voice bar shows 🕶 Lens.

🗣 Voice & wake word

Natural ElevenLabs voice (or the phone voice). Talk, Hands-free, and your own wake word (default "Hey Mentor"). Commands in English, Hindi, Arabic, Urdu, Malayalam and Tagalog: next, back, repeat, why, yes, no, not sure, confirmed, not this, stop, call expert, inspect.

Use: Hub → Settings → voice, wake word, language.

💬 Ask Ustad

Any longer sentence is a question, answered in a few spoken words from the current job (step, why, cause chain, safety, sources). Anything unsafe gets "Stop, call a professional".

Use: "*word*, why does the terminal corrode?"

🎯 Accuracy test

Pick photos where you know the real cause, choose it for each, run. Each photo goes through the real pipeline with the phone's learning switched off. Result: top-1 / top-3 accuracy, seconds per photo and a shareable report.

Use: Hub → Inspect → Accuracy test. Aim for 30+ photos.

📁 Machine history & reports

Every confirmed visit is saved per machine and can be shared as a job report. Knowledge learned on one phone can be exported and imported on another.

Use: Hub → Team → Brain & machine history.

🗒️ Consent & "What left this phone"

On first start, a plain consent screen says what Ustad sends, to whom, and what never leaves the phone. Every request (to Gemini, the backend, ElevenLabs or a local Llama) and every on-phone blur is logged with time, size and contents (e.g. "1 blurred photo + text prompt").

Use: Hub → Settings → What left this phone → Share log (for a supervisor or auditor).

🔒 Backend (keys off the phone)

A tiny Node.js server holds the Gemini and ElevenLabs keys; the app sends an app token instead. A leaked APK can be revoked without changing your paid keys. Rate-limited per token.

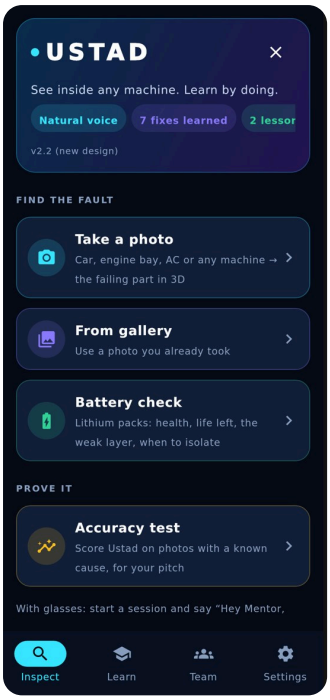
Use: deploy `render.yaml` on Render, then set `USTAD_BACKEND_URL` and `USTAD_APP_TOKEN`.

📴 Offline mode

No key or no internet: Meta's Llama 3.2 Vision runs on a laptop through Ollama and the phone uses it over USB. OpenRouter is a second online fallback.

Use: see Install, step 4.

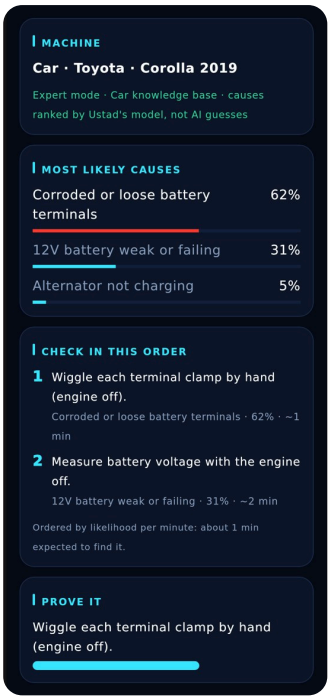
What the app looks like



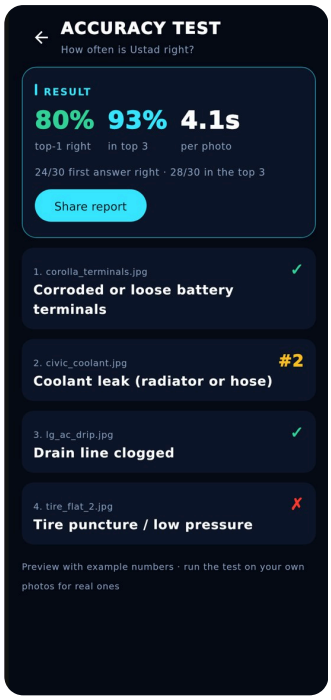
Hub · Inspect tab



Hub · Learn tab



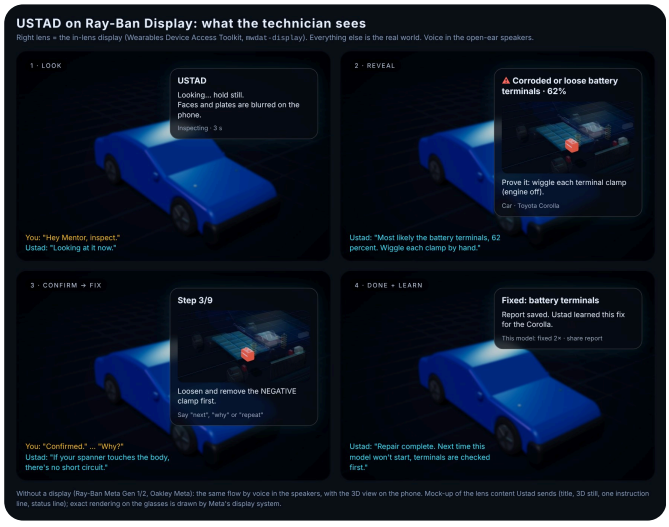
Result: causes, check order, prove it



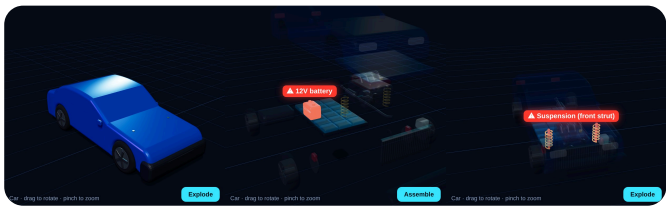
Accuracy test (example numbers)



Simple result: machine, inside view, likely causes. Everything else is under "More details".



What the Ray-Ban Display lens shows: cause, part in red, one instruction.



The 3D car: assembled and exploded.

4 Install the app

You need: Android Studio (or the Android SDK), Git, an Android phone with Android 12+ and USB debugging on, and a free Gemini API key from **aistudio.google.com/apikey**.

1 · Get the code

```
git clone https://github.com/Shaaha7/meta-1
cd meta-1/UstadApp
```

Later updates: `git pull` in the `meta-1` folder.

2 · Create `UstadApp/local.properties`

This file holds your keys and the SDK path. It is gitignored, so keys never reach GitHub.

```
sdk.dir=C\:\\Users\\YOU\\AppData\\Local\\Android\\Sdk
GEMINI_API_KEY=your_gemini_key
# optional
ELEVENLABS_API_KEY=your_elevenlabs_key
# optional: use the backend instead of keys on the phone
# USTAD_BACKEND_URL=https://your-backend.onrender.com
# USTAD_APP_TOKEN=your_token
```

Windows PowerShell shortcut for the SDK line:

```
Add-Content local.properties "sdk.dir=$($env:LOCALAPPDATA -replace '\\','\\')\\Android\\Sdk"
```

3 · Build and install

```
.\gradlew.bat installDebug        # Windows
./gradlew installDebug            # macOS / Linux
```

Or open the `UstadApp` folder in Android Studio and press Run. The version is shown at the top of the hub (it should start with **2.7**). The first start shows the privacy consent screen.

4 · Optional: offline AI on a laptop

```
ollama pull llama3.2-vision
adb reverse tcp:11434 tcp:11434
```

5 · Optional: the backend

On render.com: New → Blueprint → choose the repo (it reads `render.yaml`) → fill in `USTAD_APP_TOKENS`, `GEMINI_API_KEY`, `ELEVENLABS_API_KEY`. Put the URL and token in `local.properties` and rebuild. The free plan sleeps; open `/health` a minute before a demo.

> **Keep keys secret.** Never put keys in screenshots, chats or commits. If a key was shown anywhere, create a new one and delete the old one.

5 Using the app

A · With no glasses (the USTAD hub)

The app opens straight into the hub. The phone camera and gallery stand in for the glasses.

1. **Inspect:** Inspect tab → Take a photo (a car's engine bay, an AC, any machine) → wait a few seconds → the result shows the machine, the inside view and the likely causes.
2. **Answer and prove:** tap or say yes / no to Ustad's question; do the test; tap Prove it and photograph the result.
3. **Fix:** Start repair coach and move with “next”.
4. **Teach:** Learn tab → Record an expert (or Pick a video) → add a one-line note → Build lesson → review → Approve & publish.
5. **Learn:** Learn tab → choose a lesson → follow the steps, send checkpoint photos, take the quiz → Skill Passport.
6. **Team:** Team tab → dashboard, machine history, your expert's WhatsApp number.
7. **Settings:** language, voice (Natural / Phone), Hands-free, wake word, crash report.

B · With simulated glasses (Mock Device Kit)

1. On the glasses screen, tap the ladybug (debug) icon → Enable MockDeviceKit.
2. Pair Ray-Ban Meta, then turn on Power and Donned.
3. Choose the camera source: the phone's back camera, or an H.265 video file.
4. Start session → Preview → capture. Ustad opens with the diagnosis. Or record, stop, then “🎓 Make lesson with Ustad”.

```
ffmpeg -i job.mp4 -c:v libx265 -tag:v hvc1 -vf "scale=540:960" -an job_h265.mp4
```

C · With real Meta glasses

1. Pair the glasses in the Meta AI app and turn on Developer Mode for the Device Access Toolkit.
2. In Ustad, register the glasses and start a session. On Ray-Ban Display glasses, the lens turns on by itself.
3. Switch on Wake word (Hub → Settings) and speak: “Hey Mentor, inspect” · “next” · “why” · “Hey Mentor, check” · “Hey Mentor, call expert” · “Hey Mentor, start lesson” / “stop lesson”.

> The camera stream and voice go glasses → phone over Bluetooth. Ustad only takes a photo or records sound when you ask.

6 The demo film

File: `docs/demo/ustad-demo.mp4` in the repository (67 s, 1080p, 30 fps, voiced, with music and sound effects). It is a CGI simulation of a technician using Ustad on Meta glasses on a car that won't start.

How to show it: after `git pull`, open the file in any video player, or open it on github.com (Shaaha7/meta-1 → docs → demo → ustad-demo.mp4 → View raw / Download). Play it full-screen before the live demo, or as the backup if the network fails.



0–6 s · Title
The glasses light up; “USTAD · The master technician in your glasses”.



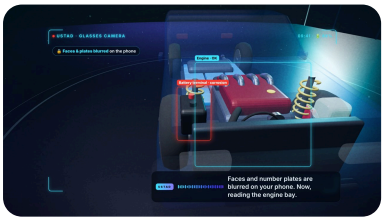
6–11 s · Ask
The technician says “Ustad, what's wrong with this car? It won't start.” The wake word is detected.



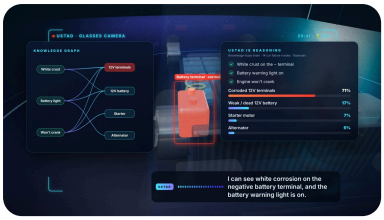
11–14 s · Capture
“Hold still. I'm taking a look.” The capture LED flashes and a beam reaches the engine bay; the camera flies into the lens.



14–19 s · Through the glasses
Capture flash, scan line, “Faces & plates blurred on the phone”, detection boxes on the battery and engine.



19–22 s · X-ray
The body and hood fade to reveal the engine bay.



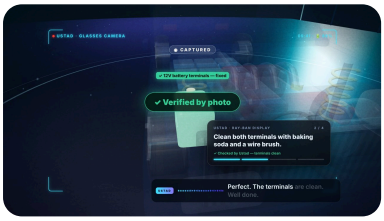
22–31 s · Reasoning
Evidence ticks in (white crust, battery light, won't crank); the knowledge graph draws itself; the causes fill to “Corroded 12V terminals 71%”.



31–40 s · Exploded view
The car comes apart, the 12V battery glows red, every part is labelled; the cause chain: acid vapour → white corrosion → high resistance → no crank.



40–46 s · Guided repair
The Ray-Ban Display card: SAFETY STOP, “Ignition off. Disconnect the negative terminal first.” The technician says “Next”.



46–51 s · Verified
Step 2, then a checkpoint photo: “✓ Verified by photo”, and the battery turns green.



51–60 s · Tutor
“One expert. Every learner.” Record → Build → Guide, and a Skill Passport badge.



60–67 s · End
“Ustad. The master technician, in your glasses.” · Inspect · Explain · Teach.

Making it photoreal or changing it

- `docs/demo/VIDEO_PROMPTS.md` has a ready prompt for each live-action shot (Veo, Sora, Kling, Runway), timed to the film. Generate the shots and cut them in over the matching seconds; keep our HUD on top.
- `docs/demo/film-source/` holds the film itself: `film.html` (three.js scene + HUD, every frame drawn by `setT(t)`), `voices.py` (open-source Kokoro voices), `audio2.py` (music and effects) and `render.js` (Playwright renders the frames; ffmpeg joins them with the sound).

7 Live demo script (3 minutes)

Time	Do	Say
0:00	Play the first 30 s of the film (or all of it).	“A technician in Dubai, a car that won't start, and no senior around.”
0:40	Hub → Inspect → photo of a real engine bay / battery terminal (or a printed photo).	“Faces and plates are blurred on the phone first. Then Ustad looks.”
1:00	Show the result: causes with %, the 3D inside view, tap Explode.	“Not an AI guess: Ustad's own knowledge graph ranks the causes, and it shows the hidden part.”
1:30	More details → How Ustad decided; answer one question by voice.	“It asks the one question that tells it the most, and checks the quickest likely cause first.”
1:50	Start repair coach; say “next”, then “why”.	“Hands-free, in English, Hindi or Arabic, with a Safety Stop built in.”
2:15	Learn tab → built-in “Clean corroded 12V battery terminals” lesson, a step and a quiz question → Skill Passport.	“One expert records once; every apprentice is coached and certified.”
2:40	Team tab → dashboard; show the accuracy-test screen.	“The supervisor sees skills and common faults, and we measure how often Ustad is right.”

Safe path: the two built-in lessons need no AI or internet. Keep the film on the laptop as a backup. Test the Wi-Fi and (if used) open the backend's `/health` one minute before you start.

8 Troubleshooting

Problem	Fix
“SDK location not found”	Add the `sdk.dir=…` line to `UstadApp/local.properties` (Install, step 2).
The app on the phone is old	`git pull`, then `.\gradlew.bat installDebug` again; the top of the hub should show 2.7.
“AI not reachable” / slow	Check the Gemini key and the internet; or use the backend; or the offline Llama mode. The first backend call on Render's free plan takes ~30 s.
Robotic voice instead of the natural one	No ElevenLabs key, no internet or no credits; Ustad falls back to the phone voice automatically.
Wake word doesn't react	Switch on Wake word in Settings, allow the microphone, speak the word then the command (“Hey Mentor, inspect”).
“No app on this phone can do that”	The phone has no app for that picker, camera or share action; install one (e.g. a gallery or WhatsApp).
“Couldn't run the privacy check”	Retake the photo; nothing was sent.
Something closed unexpectedly	Open the hub: a crash card shows the report; tap Share and send it to the developer.

9 Technology and project layout

Part	Technology
Glasses	Meta Wearables Device Access Toolkit 1.0.1: camera, mock device, display (Ray-Ban Display)
App	Android, Kotlin, Jetpack Compose
Vision AI	Gemini (inspect, lessons, checkpoints, tests, translation, Ask Ustad); OpenRouter fallback; Llama 3.2 Vision offline via Ollama
Brain	Own Bayesian knowledge-base model (naive Bayes, information gain, Dirichlet/Beta learning from confirmed cases)
Privacy	ML Kit face detection and text recognition (plates) on the phone
3D	three.js in a WebView: car, split AC, battery pack, generic layout, reasoning graph, dashboard
Voice	ElevenLabs Flash v2.5 / Eleven v3; Android speech recognition; commands in 6 languages
Backend	Node.js 18, no dependencies, app tokens and rate limits; Render or Docker
Quality	80 unit tests on the logic; Compose type-check of all screens; crash-proofing of every picker, share and glasses call

Folder	What is inside
UstadApp/	The Android app. `…/cameraaccess/ustad/` logic (brain, AI client, privacy, voice, lens, safety); `…/ui/` screens; `assets/ustad/` 3D models, knowledge bases, built-in lessons
backend/	The key-holding server (`server.js` , tests, Dockerfile)
docs/demo/	The demo film, shot prompts and film source
docs/RESEARCH_PLAN.md	Next steps: accuracy proof, live glasses inspection, more machines, real 3D models, research on sound, battery, teaching
ustad-sim/	Browser simulation of the glasses experience
render.yaml	One-click backend deploy on Render

What's next

- Run the accuracy test on 30 real fault photos and put top-3 accuracy on a slide.
- Live inspection from the glasses stream (one frame every 2 s), glasses mic for Listen.
- Pumps, motors and solar inverters as new knowledge-base modules; real part-labelled 3D models.
- Shared brain across phones, accounts and a live dashboard, hosted in the UAE (PDPL).